fevered scribblings from an unkempt mind

conor mc bride

March 11, 2026

Contents

1 beautiful abstract syntax trees are readily definable (part I) **5**
1.1 thinnings — context embeddings 6
1.2 structural term actions 7

2 depending on the traffic **9**
2.1 a universe of two-party protocols 10
2.2 a weakest-precondition semantics 11
2.3 control, response, traffic 13
2.4 dramatis personae 14
2.5 we both remember it the same 15
2.6 clemens's round 16
2.7 who's μ? 17

3 presheaves on purpose **19**

Chapter 1

beautiful abstract syntax trees are readily definable (part I)

I anticipate multiple episodes, so I can let these ideas evolve. I'm interested in first-class notions of syntax description, from which useful equipment can be constructed mechanically. Syntax with binding is a topic rich in opinion, and I am learning sufficiently rapidly that I spend a great deal of energy disagreeing with myself.

Today, I'll start from a set of *term sorts*,

$$\mathbb{T} : \mathsf{Set}$$

being the syntactic categories of some mutually recursive syntax, and grow that into a set

$$\mathbb{S} : \mathsf{Set}$$

of sorts for syntactic substructure. We'll need all the term sorts, and tupling.

$$'(T : \mathbb{T}) : \mathbb{S} \qquad 1 : \mathbb{S} \qquad (R : \mathbb{S}) \times (S : \mathbb{S}) : \mathbb{S}$$

And we'll also need binding. I'm going to make the temporary simplifying assumption that all term sorts may potentially admit binding, but we will want to refine that some other time. We can specify that we want a binding, as follows:

$$(T : \mathbb{T}) \supset (S : \mathbb{S}) : \mathbb{S}$$

The plan is then to specify a syntax by giving a proof relevant relation

$$(T : \mathbb{T}) \leftarrow (S : \mathbb{S}) : \mathsf{Set}$$

which tells you the *constructors* — all the ways to make a term of term sort T from a substructure of sort S.

Suppose we have, as contexts, backward lists of terms sorts, with a membership relation

$$(\Gamma : \mathsf{Bwd}\ \mathbb{T}) \ni (T : \mathbb{T})$$

then we can give a type of well scoped and sorted terms:

$$(\Gamma : \mathsf{Bwd}\ \mathbb{T}) \vdash (S : \mathbb{S}) : \mathsf{Set}$$

Here are the bits and pieces. Terms are made from variables and constructors.

$$\#(i : \Gamma \ni T) : \Gamma \vdash {}'T \qquad (c : T \leftarrow S) \dashv \langle (s : \Gamma \vdash S) \rangle : \Gamma \vdash {}'T$$

Tupling is achieved in the obvious way.

$$\langle\rangle : \Gamma \vdash 1 \qquad \langle (r : \Gamma \vdash R), (s : \Gamma \vdash S) \rangle : \Gamma \vdash R \times S$$

Binding extends the context.

$$\langle\rangle : \Gamma \vdash 1 \qquad \backslash (b : \Gamma, T \vdash S) : \Gamma \vdash T \supset S$$

For example, if you take $\mathbb{T} = 1$ and ensure

$$\mathsf{lam} : \langle\rangle \leftarrow \langle\rangle \supset {}'\langle\rangle \qquad \mathsf{app} : \langle\rangle \leftarrow {}'\langle\rangle \times {}'\langle\rangle$$

you get the well scoped untyped λ-terms.

1.1 thinnings — context embeddings

"Cross out *this* and write *that*!" is one of my slogans, for this and that. E.g., cross out 2 and write Prop. But here, I want to look again at binomial coefficients, except not so combinatorially. Cross out Nat and write Set! Don't just count the bloody things: say what they are and what they're for!

In this case, binomial coefficients, or 'thinnings' to their friends, can account for how everything in scope in one context stays in scope if you make the context bigger by inserting more stuff wherever you want. (Note that although one tends to insert stuff at the local end, you need only then go under a binder to find stuff even more local than the insertion, so you can't get away with mere concatenation.) Thinning types relate the 'wee end' to the 'big end', and they show in what way the big end is more.

$$(\Gamma : \mathsf{Bwd}\ X) \subseteq (\Delta : \mathsf{Bwd}\ X) : \mathsf{Set}$$

Thinning values are bit vectors, as long as the big end, and whose ones count the wee end.

$$\varepsilon : \varepsilon \subseteq \varepsilon \qquad (\theta : \Gamma \subseteq \Delta)1 : \Gamma, x \subseteq \Delta, x \qquad (\theta : \Gamma \subseteq \Delta)0 : \Gamma \subseteq \Delta, y$$

Thinnings form a category, with the empty context as initial object. Feel free to say 'semisimplicial' over and over as fast as you can.

$$\omega : \varepsilon \subseteq \Gamma \qquad \iota : \Gamma \subseteq \Gamma \qquad (\theta : \Gamma \subseteq \Delta) ; (\phi : \Delta \subseteq \Xi) : \Gamma \subseteq \Xi$$

This ω is all the 0s (0 in 2's complement binary) and this ι is all the $this$s (-1 in 2's complement binary). The composition is a kind of substitution: you substitute θ's bits for ϕ's 1s.

$$\varepsilon\,;\varepsilon = \varepsilon \qquad \theta b\,;\phi 1 = (\theta\,;\phi)b \qquad \theta\,;\phi 0 = (\theta\,;\phi)0$$

Exercise: construct a monoid on the integers whose operation is thinning composition, with neutral element -1.

One way to implement our $\Gamma \ni T$ representation of informative de Bruijn indices is as

$$\varepsilon\,, T \subseteq \Gamma$$

and I rather recommend doing this, because it then becomes clear (to us, if not to present day proof assistants) that thinnings act on terms.

$$(t : \Gamma \vdash S) \blacktriangleleft (\theta : \Gamma \subseteq \Delta) : \Delta \vdash S$$

The key observations, plain as your knows, are that

$$\#(i : \varepsilon\,, T \subseteq \Gamma) : \Gamma \vdash {}'T \qquad (\#i) \blacktriangleleft \theta = \#(i\,;\theta)$$

keeps $\Gamma \vdash S$ *strictly positive* in Γ, ensuring that action by postcomposition is achievable, and that context extension $\Gamma\,, T$ is an action on thinning *objects* which extends functorially to thinning *arrows* as $\theta 1$.

$$(\backslash b) \blacktriangleleft \theta = \backslash(b \blacktriangleleft \theta 1)$$

The rest is structural.

$$\langle\rangle \blacktriangleleft \theta = \langle\rangle \qquad \langle r, s\rangle \blacktriangleleft \theta = \langle r \blacktriangleleft \theta, s \blacktriangleleft \theta\rangle \qquad (c\text{–}\langle s\rangle) \blacktriangleleft \theta = c\text{–}\langle s \blacktriangleleft \theta\rangle$$

1.2 structural term actions

We have just seen a special case of what it is to be a structural action on terms. The message is that if you know how to cope with variable binding and usage, everything else is easy. So, let's define, *coinductively*, what it is to be an action from Γ to Δ:

$$(\Gamma : \mathsf{Bwd}\ \mathbb{T}) \triangleright (\Delta : \mathsf{Bwd}\ \mathbb{T}) : \mathsf{Set}$$

where the idea is that we are *always* ready to act on variables, and *always* ready to go under binders

$$\Gamma \triangleright \Delta = \{\mathrm{va} : \forall T.\Gamma \ni T \to \Delta \vdash {}'T;\ \mathrm{ex} : \forall T.\Gamma\,, T \triangleright \Delta\,, T\}$$

Let's write action as juxtaposition:

$$(t : \Gamma \vdash S)(\alpha : \Gamma \triangleright \Delta) : \Delta \vdash S$$

where we now have

$$(\#i)\alpha = \#(\alpha\,.\mathrm{va}\;i) \qquad (\backslash b)\alpha = \backslash(b(\alpha\,.\mathrm{ex}))$$

and, as before,

$$\langle\rangle\alpha = \langle\rangle \qquad \langle r, s\rangle\alpha = \langle r\alpha, s\alpha\rangle \qquad (c{-}\langle s\rangle)\alpha = c{-}\langle s\alpha\rangle$$

Thinnings θ yield actions $\overline{\theta}$

$$\overline{(\theta : \Gamma \subseteq \Delta)} : \Gamma \triangleright \Delta \qquad \overline{\theta} = \{\mathrm{va} = \#(\cdot\,;\theta);\ \mathrm{ex} = \overline{\theta 1}\}$$

So do substitutions. What is a substitution? Well, every context yields a sort by left-nested tupling. I propose to write that operation silently. So now $\Delta \vdash \Gamma$ is the type of left-nested tuples with a $\Delta \vdash T$ for every $\Gamma \ni T$, also known as a *simultaneous* substitution.

We shall need to *project* from these tuples. I propose first to *select* from them, using binomial coefficients to do some choosing!

$$(\theta : \Xi \subseteq \Gamma)(\sigma : \Delta \vdash \Gamma) : \Delta \vdash \Xi$$

where

$$\varepsilon\langle\rangle = \langle\rangle \quad (\theta 1)\langle\sigma, t\rangle = \langle\theta\sigma, t\rangle \qquad (\theta 0)\langle\sigma, t\rangle = \theta\sigma$$

and now we may define projection as the unpacking of a singleton selection.

$$(\sigma : \Delta \vdash \Gamma)\,!(i : \Gamma \ni T) : \Delta \vdash T \qquad \sigma\,!\,i = \mathrm{let}\ \langle\langle\rangle, t\rangle = i\sigma\ \mathrm{in}\ t$$

We are now in a position to make substitutions act.

$$\overline{(\sigma : \Delta \vdash \Gamma)} : \Gamma \triangleright \Delta \qquad \overline{\sigma} = \{\mathrm{va} = \sigma\,!\,\cdot;\ \mathrm{ex} = \langle\sigma\overline{(\iota 0)}\}, \#\omega 1\rangle$$

Chapter 2

depending on the traffic

Many of my stories begin with "Once upon an unguarded moment, I said to Sam Lindley…" and this is one of them. What I said was "If you're not doing session types with dependent types, you've got to be doing them wrong.". What I meant was that earlier signals *in* a protocol can determine the later stucture *of* the protocol, and that sounds pretty fucking dependently typed to me. Sam took a certain mischievous pleasure in relaying my offhand remark to Phil Wadler, and the next thing I knew, Phil was in my office with Simon Gay, saying "Let's do dependent session types!".

Our starting point was the CP system in Phil's "Propositions as Sessions", the rules of which we inscribed on my whiteboard, before switching to a different colour of pen and trying to bolt on type dependency. This did not work, but we had a very enjoyable lunch of spicy food and spicier nerdsniping. "Linear types are session types" went the reasoning "so dependent session types must be dependent linear types.", raising the question "So what are dependent linear types?" which was, back then, a well known hard problem that I became interested in, leading to my own "I Got Plenty O' Nuttin'" as published in Phil's Festschrift, (and repaired by Bob Atkey to make Quantitative Type Theory). But by that stage, I'd figured out that dependent linear types were *not* what dependent session types should be.

To see why not, let me tell an earlier story, this time from a summer school at Heriot-Watt University. Phil was lecturing on "Propositions as Sessions", and James McKinna, sitting on my left, asked the irritatingly perceptive question "What is it that session types classify, and what is the theorem that holds by virtue of that classification?". That's actually a rather difficult question, and Phil was a little flummoxed by it, with just cause. Read Simon Gay's lovely paper, "Subtyping Supports Safe Session Substitution", also in Phil's Festschrift: Simon observes that the session subtyping literature is divided into two perfectly dual camps, who write down perfectly opposite rules, because one camp thinks session types classify *channels* while the other thinks session types classify the *processes* interacting on channels. Of course, they do both.

James's question, phrased more directly, might as well have been "What

are you talking about?", with an implicit expletive after the "What", and it is a question dependent types people should ask, because when those variables stand for earlier stuff in later types, you need to know what *actual* stuff might end up substituting for those variables. Should dependent session types depend on channels? Should they depend on processes? Should they depend on something else?

So let's play a game, you and me. We're processes. I don't know all of the rules, but it starts with me sending you a (natural) number, and the next move is you replying to me with a number. The rest of the game carries on from there. What's it like being me? I have to have a number, the number I send you. What's it like being you? You have to have a *function* to compute your reply from my number. But you don't send me your function, just *one* of its outputs. I can't go exploring what else you might have said if I'd chosen differently, so neither would it make sense of the protocol could. Your function is a secret from me, and from the rules of the game. What it does make sense for the protocol to depend on is the pair of numbers exchanged: we both know both of them, albeit for different reasons. That's to say that what it makes sense for dependent session types to depend on is the *traffic* — the history of the signals exchanged — rather than the processes who exchange them or the channels by which they may do so. And it must be the case that that signals traffic is also something characterized by the session type.

To remark, briefly, upon dependent linear types, it's a basic Curry-Howardist position that I should like to be able to prove linearly typed processes satisfy specifications, so it is just as vital that dependent linear types support dependency on processes, as it is that dependent session types do not. I think that observation shoves more than a cigarette paper between the two notions we mistakenly conflated, back in my office. I've thought quite hard about both of them, now that I've had the clarity to separate them. I've spent less ink on dependent session types. It's time I fixed that.

2.1 a universe of two-party protocols

Let's cook up a notion of 'straight-line' protocol. I shall presume a universe of *signal types*, by which I mean types of transmissible data.

$$\mathbb{S} : \mathsf{Set} \qquad [\![\cdot]\!]_{\mathbb{S}} : \mathbb{S} \to \mathsf{Set}$$

If you chose $\mathbb{S} = 1$, $[\![\langle\rangle]\!]_{\mathbb{S}} = 2$, I should not call you a fool, but we need not be so austere. A more interesting question is whether signals in this sense should have bounded bit width. Are the natural numbers a signal type? Of course they are! Numbers fit in 64 bits, don't they?

So now our mission is to construct a universe of protocols built from signals

$$\mathbb{P} : \mathsf{Set} \qquad [\![X : \mathbb{P}]\!]_{\mathbb{P}} : \mathsf{Set}$$

where tells you the protocol's type of signal traffic. Let's do this by Dybjer and

Setzer's *induction-recursion*, allowing us to talk about traffic while we're in the process of talking about protocols.

Gently does it, though! Let's start by saying something simpler, namely what it is to be a mere *record* of signals. It's important to notice that the traffic is not any old type, but indeed, a record of signals!

$$\mathbb{R} : \mathsf{Set} \qquad [\![X : \mathbb{R}]\!]_{\mathbb{R}} : \mathsf{Set}$$

A record can store one signal, and we also have the structure required to form arbitrary dependent tuples.

$$\begin{array}{rrl} !(S : \mathbb{S}) : \mathbb{R} & [\![!S]\!]_{\mathbb{R}} = [\![S]\!]_{\mathbb{S}} & \text{store} \\ 1 : \mathbb{R} & [\![1]\!]_{\mathbb{R}} = 1 & \text{skip} \\ \Sigma\ (V : \mathbb{R})\ (W : [\![V]\!]_{\mathbb{R}} \to \mathbb{R}) : \mathbb{R} & [\![\Sigma\ V\ W]\!]_{\mathbb{R}} = (v : [\![V]\!]_{\mathbb{R}}) \times [\![W\ v]\!]_{\mathbb{R}} & V \text{ then } W \end{array}$$

Now, for protocols, we need all the same stuff and something more. We have *two* parties now, so we need a way to flip whose turn it is to send. Let's define

$$\mathbb{P} : \mathsf{Set} \qquad (X : \mathbb{P})^* : \mathbb{R} \qquad [\![X]\!]_{\mathbb{P}} = [\![X^*]\!]_{\mathbb{R}}$$

All we need add is that switching who's sending has no impact on the structure of signal traffic.

$$\begin{array}{rrl} !(S : \mathbb{S}) : \mathbb{P} & !S^* = [\![S]\!]_{\mathbb{S}} & \text{send a signal} \\ (X : \mathbb{P})^{\perp} : \mathbb{P} & X^{\perp *} = X^* & \text{swap} \\ 1 : \mathbb{P} & 1^* = 1 & \text{skip} \\ \Sigma\ (V : \mathbb{P})\ (W : [\![V]\!]_{\mathbb{P}} \to \mathbb{P}) : \mathbb{P} & (\Sigma\ V\ W)^* = \Sigma\ V^*\ (W\ \cdot)^* & V \text{ then } W \end{array}$$

And that's my point: in 'V then W', it makes sense to allow the subsequent W to depend on the traffic record from V. The traffic for the whole the the dependent pair of the traffic for both.

So, we've said what it is to *be* a protocol, but what is it to be a process which *follows* a protocol? I have two answers to that question, and some learning occurred between one and the other, so let's have both.

2.2 a weakest-precondition semantics

My first attempt to interpret protocols as types of processes gave a kind of proof-relevant weakest-precondition semantics, where each of the parties to protocol P has some *goal* $G : [\![P]\!]_{\mathbb{P}} \to \mathsf{Set}$ expressing what, to them, about the traffic in a P interaction would make them H for happy. We thus need to define (mutually) the parts in our drama

$$\begin{array}{l} \mathrm{Protagonist}\ (P : \mathbb{P})\ (G : [\![P]\!]_{\mathbb{P}} \to \mathsf{Set}) : \mathsf{Set} \\ \mathrm{Antagonist}\ (P : \mathbb{P})\ (H : [\![P]\!]_{\mathbb{P}} \to \mathsf{Set}) : \mathsf{Set} \end{array}$$

to say what is the precondition for each to achieve their goals, G and H respectively, by interacting in accordance with P. And to come good on that, we shall need to show how the drama unfolds:

$$\mathsf{synthesis}\,(P : \mathbb{P})\,(g : \mathsf{Protagonist}\,P\,G)\,(h : \mathsf{Antagonist}\,P\,H) : (p : [\![P]\!]_{\mathbb{P}}) \times G\,p \times H\,p$$

which is to say that the entire protocol runs, generating a complete record of its traffic, and both parties are satisfied with the outcome.

For our signal apparatus, we have

$$\mathsf{Protagonist}\ !S\ G = (s : S) \times G\ S \qquad \mathsf{Antagonist}\ !S\ H = (s : S) \to H\ S$$
$$\mathsf{synthesis}\ !S\ \langle s, g\rangle\ h = \langle s, \langle g, h\ s\rangle\rangle$$
$$\mathsf{Protagonist}\ P^{\perp}\ G = \mathsf{Antagonist}\ P\ G \qquad \mathsf{Antagonist}\ P^{\perp}\ H = \mathsf{Protagonist}\ P\ H$$
$$\mathsf{synthesis}\ P^{\perp}\ g\ h = \mathsf{let}\ \langle p, \langle h', g'\rangle\rangle = \mathsf{synthesis}\ P\ h\ g\ \mathsf{in}\ \langle p, \langle g', h'\rangle\rangle$$

So, your functional structure from our little game shows up in the processes, as the receiver of a signal needs a strategy to be happy, whatever it is, while the sender must choose wisely on their own part. Meanwhile, for $P^{\perp}$, we literally exchange roles!

Now, for tuple structure, we have

$$\mathsf{Protagonist}\ 1\ G = G\ \langle\rangle \qquad \mathsf{Antagonist}\ 1\ H = H\ \langle\rangle$$
$$\mathsf{synthesis}\ (1)\ g\ h = \langle\langle\rangle, \langle g, h\rangle\rangle$$
$$\mathsf{Protagonist}\ (\Sigma\ P\ Q)\ G = \mathsf{Protagonist}\ P\ \lambda p.\ \mathsf{Protagonist}\ (Q\ p)\ \lambda q.\ G\ \langle p, q\rangle$$
$$\mathsf{Antagonist}\ (\Sigma\ P\ Q)\ H = \mathsf{Antagonist}\ P\ \lambda p.\ \mathsf{Antagonist}\ (Q\ p)\ \lambda q.\ H\ \langle p, q\rangle$$
$$\begin{aligned}\mathsf{synthesis}\ (\Sigma\ P\ Q)\ g\ h = \ & \mathsf{let}\ \langle p, \langle g', h'\rangle\rangle = \mathsf{synthesis}\ P\ g\ h\ \mathsf{in}\\ & \mathsf{let}\ \langle q, \langle g'', h''\rangle\rangle = \mathsf{synthesis}\ (Q\ p)\ g'\ h'\ \mathsf{in}\\ & \langle\langle p, q\rangle, \langle g'', h''\rangle\rangle\end{aligned}$$

So the trick is to make P's postcondition for traffic p exactly the computed precondition for $Q\ p$. When we play out P, we compute exactly the processes ready to play Q.

I told this story at a Scottish Programming Languages Seminar: I remember it was at Heriot-Watt, but not exactly when. I even managed to give an example where one party sends the dimensions of a bitmap they would like to receive, then the other party sends a suitably sized matrix of bits. That's exactly the sort of thing that's hard to do in 'classic' session types, where whenever a bit is transmitted, the protocol must fork *immediately* on that bit, or *not at all*. It's progress to loosen the timing for allowing the protocol to vary with the traffic.

But I don't like this semantics, because it's not compositional enough. Whilst protocols $P : \mathbb{P}$ allow arbitrary left- and right-nesting of subprotocols, our $\mathsf{Protagonist}\,P\,G$ and $\mathsf{Antagonist}\,P\,H$ flatten that structure tail-recursively into a right-nested sequence of quantifiers. I don't mind compilers writing programs in continuation passing style, but I'm damned if I'll do it myself.

2.3 control, response, traffic

Over the years, I've learned a lot from Peter (Hank) Hancock, particularly about using dependent types to characterize interaction. "What would Hank do?" I asked myself. Hank has a particular idiolect, which I may yet teach you, but I certainly don't expect to come as standard. The Hank in my head says that a *party* to protocol $X : \mathbb{P}$ is something in

$$\mathrm{Fam}^2\ [\![X]\!]_{\mathbb{P}}$$

where

$$\mathrm{Fam}\ (S : \mathsf{Set}) = (I : \mathsf{Set}) \times (I \to S)$$

which is the *fibrational* way to talk constructively about subsets of S, by picking them out as the image of a function from some domain I. The 'squaring' in this instance indicates iterated composition (not the trigonometric usage, at all), so we get

$$\mathrm{Fam}^2\ [\![X]\!]_{\mathbb{P}} = (\mathit{Control} : \mathsf{Set}) \times \mathit{Control} \to (\mathit{Response} : \mathsf{Set}) \times \mathit{Response} \to [\![X]\!]_{\mathbb{P}}$$

But I want to say something slightly more subtle. I'm expecting *Control* to be some higher-order type of cogitations, but the *Response* had better be transmissible. Let us also do a bit of lambda-lifting. Let us say what bunch of stuff it is to be a party to an interaction delivering a value in T, which will become the traffic.

$$\begin{array}{lll} \mathsf{Party}\ (T : \mathsf{Set}) = & \{ & \mathrm{Control} \quad : \mathsf{Set} \\ & ; & \mathrm{Response} : \mathrm{Control} \to \mathbb{R} \\ & ; & \mathrm{traffic} \quad\ : (c : \mathrm{Control}) \to [\![\mathrm{Response}\ c]\!]_{\mathbb{R}} \to T \\ & \} & \end{array}$$

I gave a talk about this in a small seminar room at the University of Glasgow where I drew two enormous Easter Island heads on the whiteboard, with thought bubbles and speech bubbles to distinguish what was a cognitive process and what was an utterance. A year later, the heads were still on the board! What's important is that the traffic field determines this party's *perception* of the traffic, from what they *thought* and what they *heard*. The other party will experience the traffic from what's in their head, and what this party *says*.

If we're sensible, we should show that the parties admit tupling structure. The unit case is simple

$$\mathrm{P1} : \mathsf{Party}\ 1 \qquad \mathrm{P1}\ .\mathrm{Control} = 1; \quad \mathrm{P1}\ .\mathrm{Response}\ \langle\rangle = 1; \quad \mathrm{P1}\ .\mathrm{traffic}\ \langle\rangle\ \langle\rangle = \langle\rangle$$

For pairing, we need

$$\mathrm{P}\Sigma\ (A : \mathsf{Party}\ S) \to (B : (s : S) \to \mathsf{Party}\ (T\ s)) \to \mathsf{Party}\ ((s : S) \times T\ s)$$

It's in the Control component that higher-order structure, like that function in your head, shows up.

$$\begin{aligned}\mathrm{P}\Sigma\ A\ B\ .\mathrm{Control} = {} & (aC : A\ .\mathrm{Control})\times \\ & (aR : A\ .\mathrm{Response}\ aC) \to \\ & \mathrm{let}\ at = A\ .\mathrm{traffic}\ aC\ aR\ \mathrm{in} \\ & B\ at\ .\mathrm{Control}\end{aligned}$$

You get to *think* about the A response before you choose the B control. The response is the dependent pair of an A response and a B response to the B control computed by your function.

$$\begin{aligned}\mathrm{P}\Sigma\ A\ B\ .\mathrm{Response}\ \langle aC, bK\rangle = {} & \Sigma\ (A\ .\mathrm{Response}\ aC)\ \lambda aR. \\ & \mathrm{let}\ at = A\ .\mathrm{traffic}\ aC\ aR\ \mathrm{in} \\ & B\ at.\mathrm{Response}\ (bK\ aR)\end{aligned}$$

Finally, the traffic is the only thing that fits!

$$\begin{aligned}\mathrm{P}\Sigma\ A\ B\ .\mathrm{traffic}\ \langle aC, bK\rangle\ \langle aR, bR\rangle = {} & \mathrm{let}\ at = A\ .\mathrm{traffic}\ aC\ aR\ \mathrm{in} \\ & \mathrm{let}\ bt = B\ at\ .\mathrm{traffic}\ (bK\ aR)\ bR\ \mathrm{in} \\ & \langle at, bt\rangle\end{aligned}$$

2.4 dramatis personae

Now, let us define

$$\mathrm{Protagonist}\ (P : \mathbb{P}) : \mathsf{Party}\ [\![P]\!]_{\mathbb{P}} \qquad \mathrm{Antagonist}\ (P : \mathbb{P}) : \mathsf{Party}\ [\![P]\!]_{\mathbb{P}}$$

Of course, we shall have to say how to make them interact, but let me leave that to one side, for now.

First, we shall need parties to send and receive individual signals

$$\mathrm{P!}\ (S : \mathbb{S}) : \mathsf{Party}\ [\![S]\!]_{\mathbb{S}} \qquad \mathrm{P?}\ (S : \mathbb{S}) : \mathsf{Party}\ [\![S]\!]_{\mathbb{S}}$$

Spot the difference!

$$\begin{aligned}&\mathrm{P!}\ S\ .\mathrm{Control} = [\![S]\!]_{\mathbb{S}} && \mathrm{P?}\ S\ .\mathrm{Control} = 1 \\ &\mathrm{P!}\ S\ .\mathrm{Response}\ s = 1 && \mathrm{P?}\ S\ .\mathrm{Response}\ \langle\rangle = !S \\ &\mathrm{P!}\ S\ .\mathrm{traffic}\ s\ \langle\rangle = s && \mathrm{P?}\ S\ .\mathrm{traffic}\ \langle\rangle\ s = s\end{aligned}$$

Now we're good to go!

$$\begin{aligned}\mathrm{Protagonist}\ !S &= \mathrm{P!}\ S & \mathrm{Antagonist}\ !S &= \mathrm{P?}\ S \\ \mathrm{Protagonist}\ X^{\perp} &= \mathrm{Antagonist}\ X & \mathrm{Antagonist}\ X^{\perp} &= \mathrm{Protagonist}\ X \\ -\mathrm{tagonist}\ 1 &= \mathrm{P1} \\ -\mathrm{tagonist}\ (\Sigma\ V\ W) &= \mathrm{P}\Sigma\ (-\mathrm{tagonist}\ V)\ (-\mathrm{tagonist}\ (W\ \cdot))\end{aligned}$$

Note also that the control types for protagonist and antagonist do not directly talk about each other. Rather, they use function types to abstract over whatever it is that they just have to put up with. As a consequence, any pair of protagonist and antagonist controls will be able to talk to each other effectively. Let's prove that.

2.5 we both remember it the same

It's quite tricky to even state what we need, here. Informally, given some $X : \mathbb{P}$, we ask for the Controls for $\mathrm{Protagonist}\ X$ and $\mathrm{Antagonist}\ X$ — what's in their heads — and in return we promise to deliver both responses — what they each say out loud — and, moreover, to prove that both parties agree on what the traffic has been, even though they came by that information differently. The latter is crucial for $\Sigma\ V\ W$, because unless both parties agree on the traffic in the V-interaction, they won't even realise they're participating in the same W-protocol.

We're going to need some equations, and some kind of heterogeneity is inevitable. We have

$$\mathsf{refl} : x = x$$

but let's also have

$$(p_0 : P\ x_0) =[\mathrm{P}|(\mathrm{q} : \mathrm{x}_0 = \mathrm{x}_1)]= (p_1 : P\ x_1) : \mathsf{Set}$$

which computes to an ordinary equation when q is trivial

$$p_0 =[\mathrm{P}|\mathsf{refl}]= p_1 \quad = \quad p_0 = p_1$$

One useful thing we can do with this machinery is plug pairs together:

$$\langle(\mathrm{a_q} : \mathrm{a}_0 = \mathrm{a}_1), (\mathrm{b_q} : \mathrm{b}_0 =[\mathrm{B}|\mathrm{a_q}]= \mathrm{b}_1)\rangle : \langle a_0, b_0\rangle = \langle a_1, b_1\rangle \qquad \langle\mathsf{refl}, \mathsf{refl}\rangle = \mathsf{refl}$$

In general, we might be working with protocols which are only *provably* equal.

$$\begin{aligned}
&\mathrm{interact} : \\
&\quad (X, Y : \mathbb{P}) \to (Q : X = Y) \to \\
&\quad (cX : \mathrm{Protagonist}\ X\ .\mathrm{Control}) \to (cY : \mathrm{Antagonist}\ Y\ .\mathrm{Control}) \to \\
&\quad (\langle rX, rY\rangle : \mathrm{Protagonist}\ X\ .\mathrm{Response}\ cX \times \mathrm{Antagonist}\ Y\ .\mathrm{Response}\ cY) \times \\
&\quad \mathrm{Protagonist}\ X\ .\mathrm{traffic}\ cX\ rX =[[\![\cdot]\!]_{\mathbb{P}}|\mathrm{Q}]= \mathrm{Antagonist}\ Y\ .\mathrm{traffic}\ cY\ rY
\end{aligned}$$

By working upto some equation Q, we're allowing for the way each party perceives the protocol we're running apparently splays apart. Our first move is the good kind of conversion therapy. We J the splay away and work only on the diagonal.

$$\begin{aligned}
&\mathrm{interact}\ X\ _\ \mathsf{refl}\ cPX\ cAX = \\
&\quad (? : (\langle rPX, rAX\rangle\ : \mathrm{Protagonist}\ X\ .\mathrm{Response}\ cPX \\
&\qquad\qquad \times \mathrm{Antagonist}\ X\ .\mathrm{Response}\ cAX) \times \\
&\qquad \mathrm{Protagonist}\ X\ .\mathrm{traffic}\ cPX\ rPX = \mathrm{Antagonist}\ X\ .\mathrm{traffic}\ cAX\ rAX)
\end{aligned}$$

The crucial base case is for sending:

$$\mathrm{interact}\ !S\ _\ \mathsf{refl}\ s\ \langle\rangle = \langle\langle\langle\rangle, s\rangle, \mathsf{refl}\rangle$$

The signal that is the protagonist's Control becomes the response to the Antagonist, so they both agree it's the same signal.

To switch roles, we conjugate by swapping:

$$\begin{array}{l}\mathrm{interact}\ X^{\perp}\ _\ \mathsf{refl}\ cPX\ cAX = \\ \quad \mathrm{let}\ \langle\langle rAX, rPX\rangle, q\rangle = \mathrm{interact}\ X\ _\ \mathsf{refl}\ cAX\ cPX\ \mathrm{in} \\ \quad \langle\langle rPX, rAX\rangle, \mathrm{sym}\ q\rangle\end{array}$$

where sym proves symmetry of $=$.

Doing nothing is easy:

$$\mathrm{interact}\ 1\ _\ \mathsf{refl}\ \langle\rangle\ \langle\rangle = \langle\langle\langle\rangle, \langle\rangle\rangle, \mathsf{refl}\rangle$$

The fun case is, of course, the sequential composition. We start with our feet together

$$\begin{array}{l}\mathrm{interact}\ (\Sigma\ V\ W)\ _\mathsf{refl}\ \langle cPV, kPW\rangle\ \langle cAV, kAW\rangle = \\ \quad \mathrm{let}\ \langle\langle rPV, rAV\rangle, qV\rangle = \mathrm{interact}\ V\ _\ \mathsf{refl}\ cPV\ cAV\ \mathrm{in} \\ \quad ?\end{array}$$

We've got to the middle, and we have

$$qV : \mathrm{Protagonist}\ V\ .\mathrm{traffic}\ cPV\,rPV = \mathrm{Antagonist}\ V\ .\mathrm{traffic}\ cAV\,rAV$$

which is to say that both parties have intensionally distinct but provably equal perceptions of the V traffic. But this is the splay we prepared for! Let us carry on!

$$\begin{array}{l}\mathrm{interact}\ (\Sigma\ V\ W)\ _\mathsf{refl}\ \langle cPV, kPW\rangle\ \langle cAV, kAW\rangle = \\ \quad \mathrm{let}\ \langle\langle rPV, rAV\rangle, qV\rangle = \mathrm{interact}\ V\ _\ \mathsf{refl}\ cPV\ cAV\ \mathrm{in} \\ \quad \mathrm{let}\ \langle\langle rPW, rAW\rangle, qW\rangle = \\ \qquad\quad \mathrm{interact}\ (W\ _)\ (W\ _)\ (W\ \$\ qV)\ (kPW\ rPV)\ (kAW\ rPV)\ \mathrm{in} \\ \quad \langle\langle\langle rPV, rPW\rangle, \langle rAV, rAW\rangle\rangle, \langle qV, \mathrm{deindex}\ \mathrm{W}\ qW\rangle\rangle\end{array}$$

where we use that equality is a congruence

$$(f : S \to T)\ \$(q : x = y) : f\ x = f\ y$$

and that we may shuffle congruence around in the dependent equality:

$$\mathrm{deindex}\ W\ (qW : tPW =[[\![\cdot]\!]_{\mathbb{P}}|\mathrm{W}\ \$\ qV]= tAW) : tPW =[[\![\mathrm{W}\ \cdot]\!]_{\mathbb{P}}|qV]= tAW$$

2.6 clemens's round

Now suppose there are *three* parties, me and two others: let's call them 'Alasdair' and 'Clemens'. I can have a long and private conversation with Alasdair about whose round it is, in which we conclude that it is Clemens's round, but if *Clemens* doesn't know it's his round, we're not getting pints.

That's to say, the two-party problem makes life easy, because both are party to every signal, either as sender or receiver, so both know the traffic completely. Once there are three parties, there are secrets. Now, we have ways to represent secrets, because the 'zero' modality of Quantitative Type Theory (which I invented while staring at an empty pint glass) allows us to mark some fields of a Response record as unavailable at run time.

My point is that that's not enough. Multi-party dependent session types should not only track who knows what (including the ability for a later signal to report an earlier one and thus share what was formerly secret), but also ensure that the protocol respects need-to-know.

2.7 who's μ?

Remember that game we played? I tell you a number then you tell me one. Let's suppose we stop doing the 64-bit cheating thing and use yer actual spuds-and-lemons numbers. That's to say you communicate a number by saying "spud" your number of times, before signing off with "lemon".

In stage one of this protocol, I'm responsible for getting to the lemon eventually, so I'm inductive, and you have to be always ready for another spud, so you're coinductive. Then in stage two, these roles reverse.

We could play a more interesting game, where you can keep asking me for numbers until you've had enough. You're responsible for eventually having enough numbers, so you're inductive, and I'm repsonsible for always being ready to send you another number if you want one, so I'm coinductive. But, *locally*, when I'm in the process of sending you a number, it's me who has to get to the lemon eventually.

That's to say, in any given subprotocol, one party is responsible for the eventual termination of that subprotocol, they're the μ (least fixpoint) amongst νs (greatest fixpoint), but that responsibility can be pushed on the stack when we enter a subsubprotocol and recovered later.

In general, to extend the notio of dependent session type I've been talking about to *recursive* protocols, we'll need a fixpoint operator in the protocol language. That will cash out as a first order inductive type in the traffic. But to ensure that the traffic is indeed finite, we shall have to ensure that the protagonist's Control is a higher-order inductive type, and that the antagonists' Controls are higher-order *coinductive* types. That is to say, the protagonist when we enter a subprotocol is responsible for ensuring we leave it.

As a consequence, we now see that Control types for recursive sessions must allow arbitrary interleavings of μ and ν, in a hierarchy which we know from Julian Bradfield does not collapse. No current implementation of type theory gives effective support for these gnarly modes of recursion. The game's a bogey!

(And indeed, the type theorists in MSP know we're not going to lure Clemens Kupke into our clutches until he can have modal μ-calculus fun, and we're nowhere near making that happen.)

Chapter 3

presheaves on purpose

We love to define indexed inductive families, but so far, we have given thought only to index *values*. We can transport indexed data only across *equations* between indices. For Heaven's sake, cross out Set and write Cat, already! Ask what structure on indices the thus indexed sets respects. Ensure that we construct *functors* into Set, i.e., *presheaves* in useful and interesting ways.

We can write down a *universe* of descriptions for indexed datatypes. Now let's try to write down a universe of descriptions for presheaves-by-construction.